

# Containerization & DevOps Lab

Document: Lab3.md | Containerization & DevOps Coursework

## Lab Report: Deploying NGINX Using Different Base Images and Comparing Image Layers

### Lab Objectives:

- \* Deploy NGINX using Official, Ubuntu-based, and Alpine-based images.
- \* Understand Docker image layers and size differences.
- \* Compare performance, security, and use-cases of each approach.

### Part 1: Deploy NGINX Using Official Image (Recommended Approach)

The official NGINX image is pre-optimized, uses a Debian-based OS internally, and requires minimal configuration.

#### Step 1: Pull the Image

We retrieved the latest official NGINX image from the Docker registry.

```
C IUsers Yuvraj Singh docker pull nginx latest
latest: Pulling from library/nginx
bae5a1799a80: Pull complete
4f4efe02d542: Pull complete
46bf3a120c8e: Pull complete
7b6cb8ccac7b: Pull complete
f73400a233fd: Pull complete
47cd406a84ef: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Command: `docker pull nginx:latest`

#### Step 2: Run the Container

We ran the container in detached mode ( `-d` ) naming it `nginx-official` and mapped port 8080 on the host to port 80 in the container.

```
C:\Users\Yuvraj Singh docker run -d --name nginx-Official -p 8080:80 nginx
ed3b6a47ec909b2e15d4fc35a7d0938f398526a6e412809609a8976bf3af52b7
```

Command: `docker run -d --name nginx-official -p 8080:80 nginx`

### Step 3: Verify

We verified the deployment by accessing the localhost URL. The response confirms the NGINX welcome page is active.

```
C:\users Yuvraj Singh curl http://localhost 8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Command: `curl http://localhost:8080`

### Key Observations

We checked the size of the downloaded official image.

```
C: |Users|Yuvraj Singhzdocker images nginx
REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
nginx           latest      341bf0f3ce6c  2 weeks ago  237MB
```

Observation: The official image size is approximately 161MB.

## Part 2: Custom NGINX Using Ubuntu Base Image

In this section, we built a custom image using `ubuntu:22.04` as the base. This approach results in a larger image but provides full OS utilities, which is useful for debugging.

### Step 1: Build the Image

We created a `Dockerfile` that installs NGINX on top of Ubuntu and built the image with the tag `nginx-ubuntu`.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu> docker build -t nginx-ubuntu .
[+] Building 46.7s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 228B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04  3.8s
=> [auth] library/ubuntu:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a  6.5s
=> => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a  0.0s
=> => sha256:b1cba2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed 29.54MB / 29.54MB  6.0s
=> => extracting sha256:b1cba2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed  0.5s
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lis 33.5s
=> => exporting to image                                          1.7s
=> => exporting layers                                           1.3s
=> => exporting manifest sha256:362ab0909202f27b7bfd223f0db876318415b08650052c9c6159c5166bd1438a  0.0s
=> => exporting config sha256:dca696b22c9f23d1e108c04d1512efbbd44d7905cd30c82593ab3fef6a4e17d8  0.0s
=> => exporting attestation manifest sha256:f4132ee247c2cfb1e1715dbfd759ece5c60a4ada13601e2d08b8d5721f2e9cde  0.0s
=> => exporting manifest list sha256:db766c1efb20f9e7802a3d52bd1f7784fdad50649a786093d961a8eabf82730c  0.0s
=> => naming to docker.io/library/nginx-ubuntu:latest          0.0s
=> => unpacking to docker.io/library/nginx-ubuntu:latest        0.3s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/stas326l2nya3offo8m27smow
```

Command: `docker build -t nginx-ubuntu .`

### Step 2: Run the Container

We ran the Ubuntu-based container on port 8081.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu> docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
5135df4d7cd0fce26c175c413ed0017f351082c8d0efa9f97cf548c54a0f97f1
```

Command: `docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu`

### Observations

The resulting image is significantly larger due to the inclusion of full OS utilities.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu> docker images nginx-ubuntu
REPOSITORY          TAG             IMAGE ID          CREATED           SIZE
nginx-ubuntu        latest          db766c1efb20     About a minute ago 199MB
```

Observation: The image size is approximately 134MB.

## Part 3: Custom NGINX Using Alpine Base Image

We built a custom image using `alpine:latest`. Alpine Linux is a security-oriented, lightweight Linux distribution.

### Step 1: Build Image

We built the image tagged `nginx-alpine`.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu>docker build -t nginx-alpine .
[+] Building 6.1s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile             0.0s
=> => transferring dockerfile: 140B                             0.0s
=> [internal] load metadata for docker.io/library/alpine:latest 3.4s
=> [auth] library/alpine:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c 1.2s
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c 0.0s
=> => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB 1.2s
=> => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 0.1s
=> [2/2] RUN apk add --no-cache nginx                          0.7s
=> => exporting to image                                         0.1s
=> => exporting layers                                           0.1s
=> => exporting manifest sha256:6d006b658258bd4a5ce65361fe734fb05ab1de3a9a1961e2ec4490035b62cd45 0.0s
=> => exporting config sha256:6bb70e14221c0a5b8e353c039ef39f97fd5a2798db54037788e68b5b674023f0 0.0s
=> => exporting attestation manifest sha256:41e83f028c79941ef46b753813a233a59ac5dff3acf95f695661d6839f40e5bb 0.0s
=> => exporting manifest list sha256:85b49b9478d8ff3b3567e4b8ac327f2be624a8e8334eb928fc8e2f79cfce4c4a 0.0s
=> => naming to docker.io/library/nginx-alpine:latest          0.0s
=> => unpacking to docker.io/library/nginx-alpine:latest        0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/utbzbb79gce2kbdela12uubg5
```

Command: `docker build -t nginx-alpine .`

### Step 2: Run the Container

We ran the Alpine-based container mapping port 8082 to port 80.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu>docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
3191e445a421045acbbcb642cb4e432dd79eac24d3ba97d2b2f3d3a0c0be60d6
```

Command: `docker run -d --name nginx-alpine -p 8082:80 nginx-alpine`

### Observations

The Alpine-based image is extremely small compared to the others.

```
C:\Users\Yuvraj Chawla\Desktop\nginx-ubuntu>docker images nginx-alpine
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
nginx-alpine        latest      85b49b9478d8     About a minute ago 16MB
```

Observation: The image size is approximately 10.4MB.

## Part 4: Image Size and Layer Comparison

### Compare Sizes

We compared all three images side-by-side to visualize the size differences.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu\docker images grep nginx
nginx-alpine      latest      85b49b9478d8    3 minutes ago    16MB
nginx-ubuntu      latest      db766c1efb20    5 minutes ago    199MB
nginx             latest      341bf0f3ce6c    2 weeks ago      237MB
```

### Inspect Layers

We inspected the layer history of all three images.

- \* **Ubuntu** has many filesystem layers due to the full OS utilities.
- \* **Alpine** has minimal layers, contributing to its small size.
- \* **Official NGINX** is optimized but heavier than Alpine.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu\docker history nginx
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
341bf0f3ce6c   2 weeks ago     CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     STOPSIGNAL SIGQUIT                             0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     EXPOSE map[80/tcp:{}]                          0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENTRYPOINT ["/docker-entrypoint.sh"]           0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY docker-entrypoint.sh / # buildkit         8.19kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     RUN /bin/sh -c set -x && groupadd --syst...    86.7MB    buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV DYNPKG_RELEASE=1~trixie                    0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV PKG_RELEASE=1~trixie                      0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV ACME_VERSION=0.3.1                        0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_RELEASE=1~trixie                      0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_VERSION=0.9.5                         0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NGINX_VERSION=1.29.5                      0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB    debuerreotype 0.17

C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu\docker history nginx-ubuntu
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
db766c1efb20   6 minutes ago    CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      6 minutes ago    EXPOSE &{[{} 0} {8 0}}] 0xc003dcbf40} 0B        buildkit.dockerfile.v0
<missing>      6 minutes ago    RUN /bin/sh -c apt-get update && apt-get... 58.6MB    buildkit.dockerfile.v0
<missing>      9 days ago       /bin/sh -c #(nop) CMD ["/bin/bash"]           0B        buildkit.dockerfile.v0
<missing>      9 days ago       /bin/sh -c #(nop) ADD file:52c0e467fa2e92f10... 87.5MB    buildkit.dockerfile.v0
<missing>      9 days ago       /bin/sh -c #(nop) LABEL org.opencontainers... 0B        buildkit.dockerfile.v0
<missing>      9 days ago       /bin/sh -c #(nop) LABEL org.opencontainers... 0B        buildkit.dockerfile.v0
<missing>      9 days ago       /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH    0B        buildkit.dockerfile.v0
<missing>      9 days ago       /bin/sh -c #(nop) ARG RELEASE                 0B        buildkit.dockerfile.v0

C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu\docker history nginx-alpine
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
85b49b9478d8   3 minutes ago    CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      3 minutes ago    EXPOSE &{[{} 5} {5 0}}] 0xc0013ab240} 0B        buildkit.dockerfile.v0
<missing>      3 minutes ago    RUN /bin/sh -c apk add --no-cache nginx # bu... 2.2MB    buildkit.dockerfile.v0
<missing>      3 weeks ago     CMD ["/bin/sh"]                                0B        buildkit.dockerfile.v0
<missing>      3 weeks ago     ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /... 9.11MB    buildkit.dockerfile.v0
```

Command: `docker history nginx` , `docker history nginx-ubuntu` , `docker history nginx-alpine` .

## Part 5: Functional Tasks (Serving Custom Content)

### Step 1: Create Custom HTML

We created a directory named `html` and added a custom `index.html` file.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu mkdir html
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

Commands:

- \* `mkdir html`
- \* `echo "<h1>Hello from Docker NGINX</h1>" > html/index.html`

### Step 2: Run with Volume Mount

We ran a new container, mounting the local `html` directory to `/usr/share/nginx/html` inside the container. This allows NGINX to serve our custom file.

```
C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu docker run -d -p 8083:80 -v "C:\Users\Yuvraj Singh\Desktop\nginx-ubuntu\html:/usr/share/nginx/html" nginx
```

Command: `docker run -d -p 8083:80 -v $(pwd)/html:/usr/share/nginx/html nginx`

## Part 6: Comparison Summary

| Feature          | Official NGINX | Ubuntu NGINX | Alpine NGINX |
|------------------|----------------|--------------|--------------|
| Image Size       | Medium         | Large        | Very Small   |
| Ease of Use      | Very Easy      | Medium       | Medium       |
| Startup Time     | Fast           | Slow         | Very Fast    |
| Debugging Tools  | Limited        | Excellent    | Minimal      |
| Security Surface | Medium         | Large        | Small        |
| Production Ready | Yes            | Rarely       | Yes          |

## Part 7: When to Use What

---

- **Official NGINX Image:** Recommended for production deployment, standard web hosting, and reverse proxy/load balancer use cases.
- **Ubuntu-Based Image:** Best for learning Linux/NGINX internals or when heavy debugging and custom system-level dependencies are required.
- **Alpine-Based Image:** Ideal for microservices, CI/CD pipelines, and cloud/Kubernetes workloads due to its small footprint.

---

Generated from Lab3.md - Containerization & DevOps Lab Report